Software Engineering for Geoinformatics

Requirement Analysis & Specification Document (RASD)

MeteoCity

Group : 9

Authors :
Ayman Abdelgadir
Mobina Faraji
Hangyun Wang

1. Introduction

1. Purpose

This document describes the Requirement Analysis and Specification Document (RASD) for a client-server application designed to support city-level heat monitoring using Meteostat data. The platform aims to help public authorities, environmental agencies, researchers, NGOs, and citizens retrieve, visualize, and analyze historical weather-station observations.

2. Scope

The system will retrieve historical meteorological observations from Meteostat, store them in a structured database, expose them through a REST API, and provide interactive visualizations through a dashboard implemented using Jupyter Notebook.

3. Requirements Gathering Process

Requirements were identified through analysis of the official project assignment, course lectures on requirement engineering, brainstorming sessions, and evaluation of Meteostat functionalities and available APIs.

2. Overall Description

1. Product Perspective

The project is composed of three main components: a database for weather observations, a backend web server exposing REST APIs, and an interactive dashboard for visualization and analysis.

2. System Boundaries

The following activities and capabilities are explicitly outside the scope of the system:

- The system does not perform real-time weather forecasting or predictive modeling.
- The system does not directly control or communicate with physical meteorological stations.
- The system does not collect raw sensor data; it relies solely on data provided by the Meteostat external API.
- The system does not support user account management or authentication natively (delegated to external identity providers).
- The system does not generate alerts or automated notifications based on weather thresholds.
- The system does not cover weather variables beyond those available through the Meteostat API.
- The system does not support real-time (live) streaming of weather observations.

2.3 External Actors

Actor	Description
Public Authorities	Analyze urban heat and weather patterns to support policy decisions
Environmental Agencies	Monitor climate-related indicators across cities
Researchers/Students	Explore historical meteorological datasets for academic purposes
NGOs	Retrieve and monitor meteorological data to support environmental and humanitarian initiatives
Meteostat API	Provides historical weather observations as an external data source

2.4 Relevant Phenomena

The relevant phenomena considered within the scope of this system include:

- Weather-station measurements (temperature, humidity, wind speed, etc.).
- Temperature changes over time across monitored cities.
- Station metadata retrieval from the Meteostat API.
- API requests sent to and responses received from Meteostat.
- User queries submitted through the dashboard interface.
- Dashboard interactions including map navigation, chart rendering, and filtering.
- Data visualization processes for meteorological indicators.
- Data export events triggered by user actions.

3. Functional Requirements

ID	Functional Requirement	Priority
FR1	The system shall retrieve historical weather observations from Meteostat.	**HIGH**
FR2	The system shall store station metadata and hourly observations in a database.	**HIGH**
FR3	The system shall allow users to query weather data by city and date range.	**HIGH**
FR4	The system shall visualize weather stations on an interactive map.	**HIGH**
FR5	The system shall display charts and summary indicators for weather observations.	**HIGH**
FR6	The system shall provide filtering capabilities for meteorological variables.	**MEDIUM**
FR7	The system shall provide interactive visualization and analysis of meteorological data.	**MEDIUM**

4. Non-Functional Requirements

ID	Non-Functional Requirement	Priority
NFR1	The dashboard response time shall remain below 5 seconds for standard queries.	HIGH
NFR2	The interface shall be intuitive and easy to use.	HIGH
NFR3	The system shall support compatibility with major browsers.	HIGH
NFR4	The system shall ensure data consistency and integrity.	HIGH
NFR5	The system must be available to the users.	HIGH

5. Technical Requirements and Constraints

ID	Technical Requirement	Priority
TR1	The system shall integrate with the Meteostat API using RapidAPI.	HIGH
TR2	The backend shall expose REST APIs.	HIGH
TR3	The dashboard must be implemented using Jupyter Notebook.	HIGH
TR4	Data shall be exchanged using JSON format.	HIGH
TR5	The database shall support structured meteorological storage.	HIGH
TR6	The system shall expose weather data through REST APIs in JSON format.	HIGH
TR7	The system shall allow users to export filtered observations as CSV files.	HIGH

6. Use Case Descriptions

Access Levels:

- Environmental Officials (G. Official): Full access – Login, Logout, Geographic Visualization, Query Data, Trend Dashboard, Statistics Summary, Filter, Export.
- NGO Representative: Login, Logout, Geographic Visualization, Query Data, Trend Dashboard, Filter, Export.
- Student: Login, Logout, Geographic Visualization, Query Data, Filter, Export.

1. Use Cases

Table 4: Environmental Official user case

Name	Analyze Regional Heat Trends
Actor	Environmental Official (G. Official)
Flow of Events	Register → Login → Geographic Visualization → Query Data → Trend Dashboard → Statistics Summary → Filter → Export → Logout
Exceptions	Data not available for selected filter.

Table 5: NGO Representative user case

Name	View Heat Pollution Dashboard
Actor	NGO Representative
Flow of Events	Register → Login → Geographic Visualization → Query Data → Trend Dashboard → Filter → Export → Logout
Exceptions	Charts fail to render.

Table 6: Student user case

Name	Explore Weather Station Map
Actor	Student
Flow of Events	Register → Login → Geographic Visualization → Query Data → Filter → Export → Logout
Exceptions	Map tiles fail to load.

7. UML Class Diagram

The following table presents the main classes, their attributes, and their methods in the system:

Class	Attributes	Methods
User	- user_id: int - username: string - role: string	+ login() + logout() + queryData(city, dateRange) + exportCSV()
WeatherStation	- station_id: string - name: string - city: string - latitude: float - longitude: float	+ getMetadata() + getObservations(dateRange) + getLocation()
Observation	- observation_id: int - station_id: string (FK) - datetime: datetime - temperature: float - humidity: float - wind_speed: float	+ getByCity(city, dateRange) + getByStation(stationId) + toCSV()
Dashboard	- charts: List[Chart] - filters: FilterConfig - activeCity: string	+ renderMap() + renderChart(variable) + applyFilter(variable, range) + exportCSV()
MeteostatAPI	- base_url: string - api_key: string	+ fetchObservations(stationId, start, end) + fetchStations(city) + parseResponse(json)

Relationships:

- User interacts with Dashboard (association).
- Dashboard queries Observation and WeatherStation (dependency).
- MeteostatAPI supplies data that is stored as WeatherStation and Observation records (dependency).
- Observation is associated with WeatherStation via station_id (foreign key / association).

8. Use Case Diagram

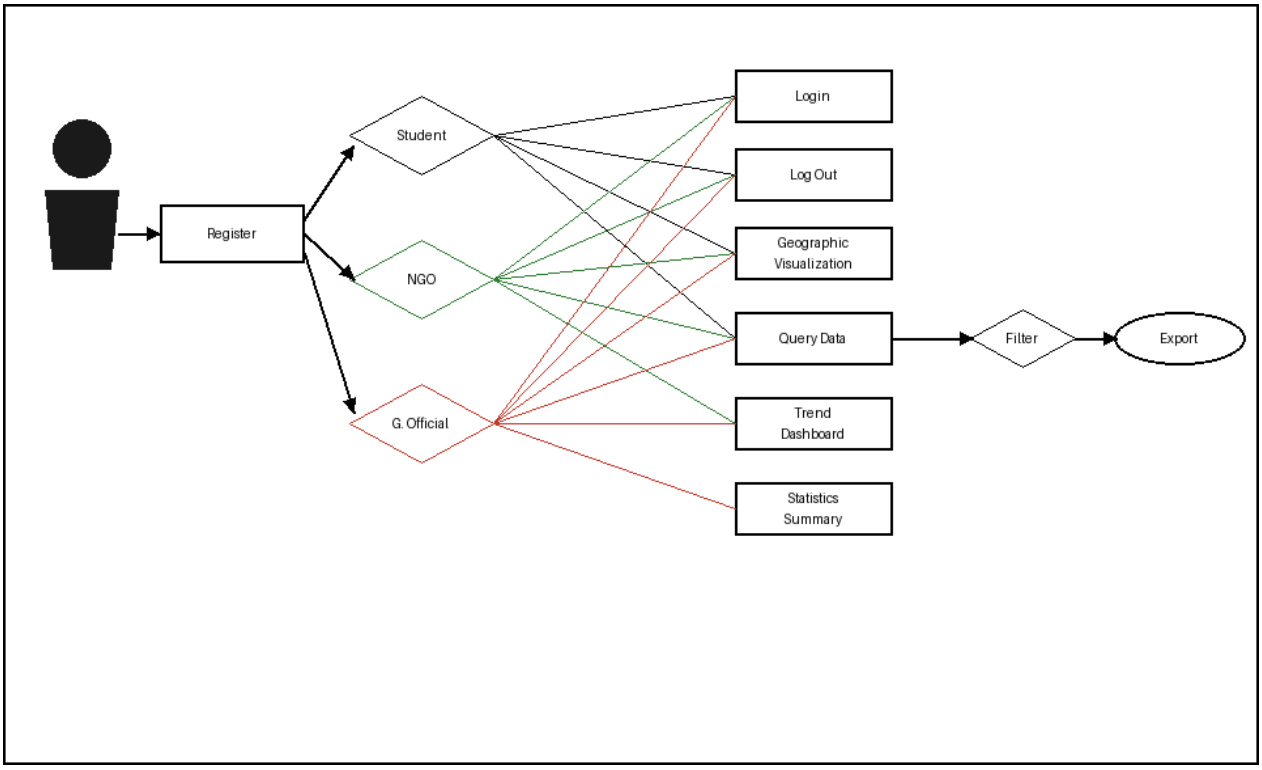


Figure 1: Use Case Diagram